

Automation Exercise – Dokumentacja

Created by: Bartłomiej Wilk (TaloHomes404)

Spis treści

1. Test Plan – Wprowadzenie	3
2. Zakres testów (Scope of Testing)	3
3. Strategia testów	4
4. Tworzenie i konfiguracja projektu	4
5. Przypadki testowe	8
6. Framework i automatyzacja przypadków testowych	14
7. Testowanie API.....	20

1. Test Plan – Wprowadzenie

Utworzony dokument ma za zadanie przedstawić proces od utworzenia nowego projektu do wdrożenia automatyzacji testów w aplikacji webowej. Jest to drugi projekt w którym chcę praktycznie wykorzystać wiedzę z zakresu podstaw automatyzacji testów w aplikacjach webowych z wykorzystaniem narzędzia Playwright oraz wszystkich narzędzi użytych w tym procesie.

Główne frameworki i narzędzia użyte w projekcie:

- **Playwright**
- **Maven**
- **Java**
- **IntelliJ**
- **JUnit 5**
- **Allure Reports**
- **CI/CD w Github Actions**
- **Git i Github**

Głównym podmiotem projektu będzie aplikacja webowa **Automation Exercise**, czyli platforma stworzona do praktycznego sprawdzenia swoich umiejętności z zakresu testów automatycznych. Strona jest dostępna pod adresem: <https://www.automationexercise.com/>

2. Zakres testów (Scope of Testing)

In Scope:

- Moduł autoryzacji użytkownika (Logowanie / Rejestracja)
- Moduł koszyka i finalizacji zamówień
- Wyszukiwarka produktów
- Nawigacja oraz poprawność wyświetlania i działania elementów UI
- Testy API

Out of scope:

- Bramki płatnicze
- Wydajność
- Testy bezpieczeństwa

3. Strategia testów

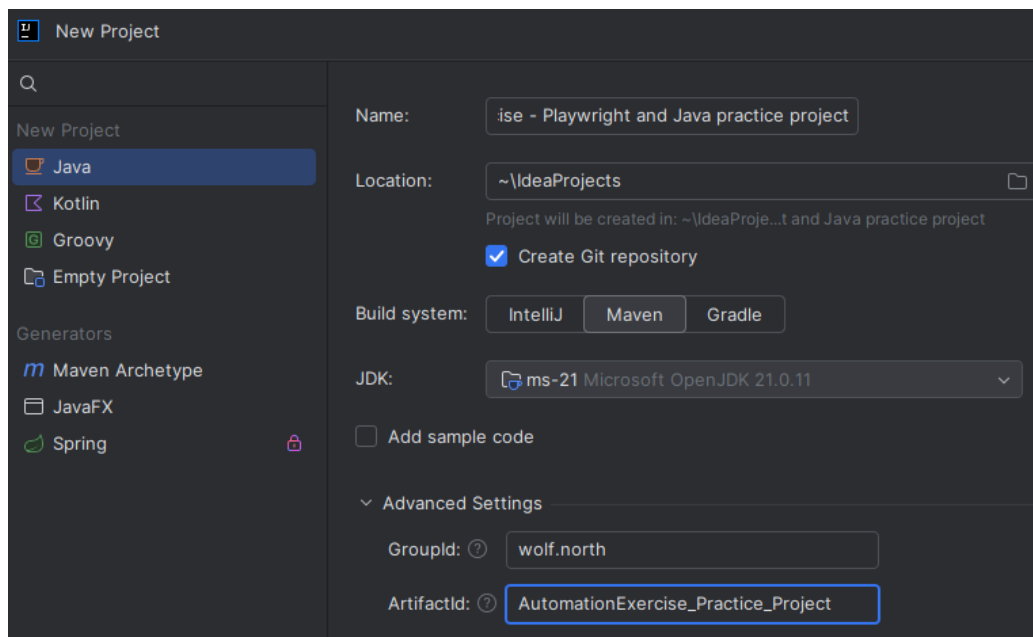
Celem będzie zapewnienie wysokiej jakości aplikacji poprzez testowanie najważniejszych modułów przed każdym wydaniem w środowisko produkcyjne. Poprzez wykorzystanie różnych rodzajów testów jak **smoke testy**, **testy E2E** czy **testy API** sprawdzimy, czy aplikacja poprawnie działa oraz wprowadzone zmiany nie sprawiły, że któryś z głównych modułów działa nieprawidłowo.

Kluczowym założeniem wykonania testów będzie ich automatyzacja, chcąc skrócić czas regresji kluczowych modułów, przypadki testowe zostaną zautomatyzowane oraz wprowadzone w środowisko **CI/CD**, aby były uruchamiane z każdą większą zmianą w kodzie.

4. Tworzenie i konfiguracja projektu

- **Konfiguracja projektu i instalacja**

Pierwszym krokiem w konfiguracji będzie utworzenie nowego projektu w środowisku IntelliJ i ustawienie wszystkich zależności w Maven, utworzenie i inicjalizacja repozytorium Git oraz poprawne ustawienia struktury projektu.



Następnie konfigurujemy główny plik narzędzia Maven o nazwie **pom.xml** dodając zależności do projektu z oficjalnego repozytorium dla Playwright, JUnit 5, Allure Reports, DataFaker oraz aktualizacja pluginu surefire. Przy okazji stworzymy i zainicjujemy repozytorium Git oraz utworzymy nowego brancha na którym skonfigurujemy projekt.

```
Bartek@DESKTOP-07KMHRM MINGW64 ~/IdeaProjects/Automation Exercise - Playwright and Java practi
$ git checkout -b config/init
Switched to a new branch 'config/init'

Bartek@DESKTOP-07KMHRM MINGW64 ~/IdeaProjects/Automation Exercise - Playwright and Java practi
$
```

```
Dependencies
> org.junit.jupiter:junit-jupiter-api:5.13.4 (test)
> io.qameta.allure:allure-junit5:2.34.0 (test)
> com.microsoft.playwright:playwright:1.59.0
> net.datafaker:datafaker:2.5.4
> org.apache.maven.plugins:maven-surefire-plugin:3.5.5
```

Następnie tworzymy strukturę katalogów oraz podstawowe pliki konfiguracyjne.

```
src
├── main
│   ├── java
│   │   ├── components
│   │   ├── config
│   │   ├── factory
│   │   ├── pages
│   │   └── utils
│   └── resources
└── test
    ├── java
    │   ├── base
    │   └── tests
```

```
public class PlaywrightFactory { 3 usages  Bartłomiej Wilk

    //Init vals for factory
    private static Playwright playwright; 5 usages
    private static Browser browser; 3 usages
    private static BrowserContext context; 3 usages
    private static Page page; 3 usages

    //Init browser and context method
    public static Page initBrowser(){...}

    public static void closeBrowser(){...}

}
```

```

public class ConfigManager { 3 usages new *

    public static final String BASE_URL = no usages
        System.getProperty(key: "env", def: "https://www.automationexercise.com/");

    public static final String BROWSER = 1 usage
        System.getProperty(key: "browser", def: "chromium");

    public static final boolean HEADLESS = 1 usage
        Boolean.parseBoolean(
            System.getProperty(key: "headless", def: "true")
        );
}

```

Jest to ważny krok, gdyż dobre praktyki mówią, aby tworzyć instancje w jednym miejscu, jest to nie dość, że łatwiejsze w konfiguracji to jeszcze umożliwia zmianę konfiguracji w jednym miejscu zamiast przeszukiwania 50 plików i manualnego zmieniania wartości w każdym wystąpieniu.

Następnie stworzyliśmy klasę w katalogu test/java/base o nazwie **BaseTest** która jest podstawowym punktem w naszym frameworku Playwright – i jest must have! To tutaj będą znajdowały się metody, które pozwolą na stworzenie punktu startowego, przeglądarki, oraz wszystkich akcji, które będą podejmowane przed/po testach, jak np. idealnym przykładem jest, że miejsce w metodzie o nazwie **tearDown** będzie tworzenie screenshota i zapisywania informacji po każdym teście np. z statusem FAIL.

- **Tworzenie struktury POM**

W celu zapewnienia odpowiednich praktyk wykorzystywanych w testowaniu automatycznym, stworzymy prostą strukturę POM w naszym projekcie, czyli utworzenie katalogów oraz modułów/stron/ekranów/komponentów z zasadą **single responsibility** które zapewnią lepszą czytelność kodu, łatwiejsze zmiany i szybsze wprowadzanie zmian/korekt w elementach UI czy flow aplikacji.

Dla przedstawienia w dokumentacji zaprezentuje klasę **LoginPage** która reprezentuje ekran logowania, w przypadku naszej strony widnieje także tam wstępna rejestracja która wymaga podania imienia i adresu e-mail oraz przenosi nas na pełną stronę logowania (która również została odwzorowana w strukturze POM w klasie **SignupPage**)

```

public class LoginPage {
    private final Page page;

    //== Locators ==//
    //Login Section
    private final Locator loginEmailField;
    private final Locator loginPasswordField;
    private final Locator loginButton;

    //Signup Section
    private final Locator usernameSignupField;
    private final Locator emailSignupField;
}

```

```

private final Locator signupButton;

public LoginPage(Page page){
    this.page = page;

    //Login
    this.loginEmailField = page.locator("[data-qa='login-email']");
    this.loginPasswordField = page.locator("[data-qa='login-
password']");
    this.loginButton = page.getByRole(AriaRole.BUTTON, new
Page.GetByRoleOptions().setName("Login"));

    //Signup
    this.usernameSignupField = page.locator("[data-qa='signup-name']");
    this.emailSignupField = page.locator("[data-qa='signup-email']");
    this.signupButton = page.getByRole(AriaRole.BUTTON, new
Page.GetByRoleOptions().setName("Signup"));
}

public void open() { page.navigate(ConfigManager.BASE_URL + "login"); }

public void login(String email, String password){
    loginEmailField.fill(email);
    loginPasswordField.fill(password);
    loginButton.click();
}

public void preRegister(String name, String email){
    usernameSignupField.fill(name);
    emailSignupField.fill(email);
    signupButton.click();
}
}

```

Jak możemy zauważyć, przedstawione rozwiązanie ma na celu odseparować elementy i logikę biznesową od plików testowych, gdzie zależy nam na sprawdzeniu poprawności funkcjonowania a nie na sposobie działania danych funkcji.

5. Przypadki testowe

- Moduł autoryzacji (logowanie / rejestracja)

Identyfikator	TC-REG01 – Poprawna rejestracja użytkownika w systemie
Priorytet	Najwyższy
Opis	Testowanie flow aplikacji poprzez rejestrację użytkownika prawidłowymi wartościami w naszym systemie.
Dane wejściowe	Dane użytkownika wygenerowane przez bibliotekę Faker
Kroki	1. Przejdź na stronę główną https://automationexercise.com/ 2. Naciśnij przycisk „Signup / Login” 3. Uzupełnij wartości z prawej strony pod napisem „New User Signup!” wartościami z danych wejściowych. 4. Kliknij przycisk „Signup” 5. Uzupełnij pola rejestracji (Title, name, email, password, date of birth) 6. Zaznacz checkboxy od news lettera i specjalnych okazji 7. Uzupełnij pola kontaktowe (First name, Last Name, Address, Country, State, City, Zipcode, Mobile number) 8. Wciśnij przycisk “Create Account”.
Oczekiwany rezultat	Użytkownik powinien zostać poprawnie zarejestrowany na stronie, po utworzeniu konta powinny się pojawić napis jak „ACCOUNT CREATED!” oraz po przejściu dalej powinny wyświetlić się podstawowe dane użytkownika.

Identyfikator	TC-REG02 – Rejestrowanie użytkownika z już istniejącymi w systemie danymi
Priorytet	Wysoki
Opis	Testowanie flow aplikacji poprzez rejestrację użytkownika danymi na które jest już utworzone konto w systemie.
Dane wejściowe	Name: Robert E-mail: robertsmith11@email.com
Kroki	1. Przejdź na stronę główną https://automationexercise.com/ 2. Naciśnij przycisk „Signup / Login” 3. Uzupełnij wartości z prawej strony pod napisem „New User Signup!” wartościami z danych wejściowych. 4. Kliknij przycisk „Signup”
Oczekiwany rezultat	Użytkownik nie powinien zostać przekierowany do dalszego kroku procesu rejestracji, komunikat błędu „Email Address already exist!” został wyświetlony użytkownikowi.

Identyfikator	TC-LOG01 – Zaloguj użytkownika podając istniejące i poprawne dane
Priorytet	Najwyższy
Opis	Testowanie flow aplikacji dla poprawnego logowania danymi które są zarejestrowane w systemie.
Dane wejściowe	E-mail: robertsmith11@email.com Password: robertsmithpass123
Kroki	1. Przejdź na stronę główną https://automationexercise.com/ 2. Naciśnij przycisk „Signup / Login”

	3. Uzupełnij pola z lewej strony pod napisem „Login to your account” wartościami z danych wejściowych. 4. Wciśnij przycisk „Login”
Oczekiwany rezultat	Użytkownik powinien zostać poprawnie zalogowany na konto, napis „Logged in as (username)” jest widoczny.

Identyfikator	TC-LOG02 – Zaloguj użytkownika podając niepoprawne dane
Priorytet	Wysoki
Opis	Walidacja poprawności pól logowania, test negatywny
Dane wejściowe	E-mail: unknownrichard@tlen.pl Password: testpassword123
Kroki	1. Przejdź na stronę główną https://automationexercise.com/ 2. Naciśnij przycisk „Signup / Login” 3. Uzupełnij pola z lewej strony pod napisem „Login to your account” wartościami z danych wejściowych. 4. Wciśnij przycisk „Login”
Oczekiwany rezultat	Użytkownik nie powinien zostać zalogowany do systemu, napis „Your email or password is incorrect!” jest widoczny.

Identyfikator	TC-LOG03 – Poprawne wylogowanie zalogowanego użytkownika
Priorytet	Średni
Opis	Weryfikowanie poprawnej funkcjonalności wylogowania użytkownika
Dane wejściowe	E-mail: robertsmith11@email.com Password: robertsmithpass123
Kroki	1. Przejdź na stronę główną https://automationexercise.com/ 2. Naciśnij przycisk „Signup / Login” 3. Uzupełnij pola z lewej strony pod napisem „Login to your account” wartościami z danych wejściowych. 4. Wciśnij przycisk „Login” 5. Wciśnij przycisk „Logout”
Oczekiwany rezultat	Użytkownik powinien zostać poprawnie wylogowany i przekierowany na stronę główną oraz „Signup / Login” wyświetla się w górnym pasku jako jedna z opcji.

- Moduł koszyka

Identyfikator	TC-CART01 – Dodanie dwóch produktów do koszyka oraz walidacja poprawności wyświetlania i obliczania zawartości
Priorytet	Najwyższy
Opis	Testowanie flow aplikacji poprzez umieszczenie produktów w koszyku oraz walidacja czy są poprawnie wyświetlane oraz obliczane.
Dane wejściowe	Niezałogowany użytkownik
Kroki	1. Przejdź na stronę główną https://automationexercise.com/ 2. Z górnego paska nawigacyjnego wybierz „Products” 3. W sekcji All Products wybierz pierwszą pozycję (Blue Top) oraz kliknij przycisk „Add to cart”. 4. W wyświetlonym komunikacie naciśnij przycisk „Continue Shopping”. 5. W sekcji all products wybierz drugą pozycję (Men Tshirt) oraz kliknij przycisk „Add to cart”. 6. W wyświetlonym komunikacie naciśnij podświetlony załącznik „View Cart”
Oczekiwany rezultat	Użytkownikowi powinna zostać wyświetlona strona koszyka, w którym znajdują się dwie pozycje, „Men Tshirt”, „Blue Top”, przy każdej pozycji pole Quantity wskazuje 1 , a cena wynosi odpowiednio Rs. 500 oraz Rs. 400 .

Identyfikator	TC-CART02 – Dodawanie kilku sztuk produktu do koszyka i walidacja poprawności wyświetlania ilości i obliczania cen.
Priorytet	Wysoki
Opis	Weryfikowanie poprawności wyświetlania i obliczania cen dla różnych ilości dodanego produktu.
Dane wejściowe	Niezałogowany użytkownik
Kroki	1. Przejdź na stronę główną https://automationexercise.com/ 2. Z górnego paska nawigacyjnego wybierz „Products” 3. W sekcji All Products wybierz trzecią pozycję (Sleeveless Dress) oraz naciśnij przycisk „View Product”. 4. W polu Quantity: zwiększ wartość do 4. 5. Wciśnij przycisk „Add to cart” 6. W wyświetlonym komunikacie naciśnij podświetlony załącznik „View Cart”
Oczekiwany rezultat	Użytkownik powinien zostać przekierowany na stronę koszyka, gdzie znajduje się tylko dodany produkt (Sleeveless Dress), pole Quantity pokazuje wartość 4 a wartość Total wynosi Rs. 4000

Identyfikator	TC-CART03 – Usuwanie produktów z koszyka.
Priorytet	Średni
Opis	Weryfikowanie poprawności wyświetlania dla procesu usunięcia produktów z koszyka.
Dane wejściowe	Niezałogowany użytkownik
Kroki	1. Przejdź na stronę główną https://automationexercise.com/ 2. W sekcji Features Items wybierz trzecią pozycję „Sleeveless Dress” oraz naciśnij przycisk „Add to cart”

	3. W wyświetlonym komunikacie naciśnij podświetlony załącznik „View Cart”. 4. Wciśnij przycisk „X” z prawej strony dodanego produktu.
Oczekiwany rezultat	Dodany produkt powinien zostać usunięty z koszyka, zostaje wyświetlony napis „Cart is empty! Click here to buy products.”

Identyfikator	TC-CART04 – Dodaj produkt do koszyka z sekcji produktów rekomendowanych.
Priorytet	Średni
Opis	Testowanie flow aplikacji czy przedmioty wyświetlane w sekcji rekomendowanych produktów poprawnie dodają się do koszyka.
Dane wejściowe	Niezałogowany użytkownik
Kroki	1. Przejdź na stronę główną https://automationexercise.com/ 2. W dolnej części strony z sekcji „Recommended Items” wybierz „Summer White Top” oraz wciśnij przycisk „Add to cart” 3. W wyświetlonym komunikacie naciśnij przycisk „View Cart”
Oczekiwany rezultat	Użytkownikowi powinna zostać wyświetlona strona koszyka, w którym znajdują się pozycja z rekomendowanych produktów o nazwie „Summer White Top” z polami Quantity 1 oraz ceną Rs. 400

Identyfikator	TC-CART05 – Dodaj produkt do koszyka ze strony szczegółów produktu.
Priorytet	Wysoki
Opis	Testowanie flow aplikacji poprawności dodawania produktów do koszyka ze strony szczegółów produktu.
Dane wejściowe	Niezałogowany użytkownik
Kroki	1. Przejdź na stronę główną https://automationexercise.com/ 2. W sekcji „Features items” wybierz czwartą pozycję „Stylish Dress” oraz wciśnij przycisk „View Product”. 3. W wyświetlonym komunikacie naciśnij przycisk „View Cart” 4. Wciśnij przycisk „Add to cart” 5. W wyświetlonym komunikacie naciśnij podświetlony załącznik „View Cart”
Oczekiwany rezultat	Użytkownikowi powinna zostać wyświetlona strona koszyka, w którym znajduje się dodana pozycja „Stylish Dress” z Quantity 1 oraz ceną Rs. 1500.

- Moduł wyszukiwania produktów

Identyfikator	TC-SEARCH01 – Wyszukiwanie prawidłowego produktu.
Priorytet	Najwyższy
Opis	Testowanie flow aplikacji poprzez wyszukiwanie produktów oraz walidacja czy został wyświetlony odpowiedni produkt.
Dane wejściowe	Niezałogowany użytkownik
Kroki	1. Przejdź na stronę główną https://automationexercise.com/ 2. Z górnego paska nawigacyjnego wybierz „Products” 3. W pole tekstowe „Search Product” wprowadź wartość „Rose” oraz wciśnij przycisk z ikoną lupy obok.
Oczekiwany rezultat	Użytkownikowi powinien zostać wyświetlony wynik wyszukiwania, gdzie w sekcji „Searched Products” znajduje się produkt „Rose Pink Embroidered Maxi Dress”

Identyfikator	TC-SEARCH02 – Wyszukiwanie nieistniejącego produktu.
Priorytet	Najwyższy
Opis	Testowanie flow aplikacji poprzez wyszukiwanie nieistniejącego produktu, negative test.
Dane wejściowe	Niezałogowany użytkownik
Kroki	1. Przejdź na stronę główną https://automationexercise.com/ 2. Z górnego paska nawigacyjnego wybierz „Products” 3. W pole tekstowe „Search Product” wprowadź wartość „Kłapa gaźnika matiz 0.8 e” oraz wciśnij przycisk z ikoną lupy obok.
Oczekiwany rezultat	Użytkownikowi zostaje przedstawiona strona wyszukiwania, nie wyświetlają się żadne produkty.

- Moduł realizacji zamówień (Checkout)

Identyfikator	TC-CHECKOUT01 – Checkout zamówionych produktów jako poprawnie zalogowany użytkownik
Priorytet	Najwyższy
Opis	Testowanie flow aplikacji poprzez testowanie czy poprawnie zarejestrowany i zalogowany użytkownik może sfinalizować zamówienie poprzez koszyk.
Dane wejściowe	E-mail: robertsmith11@email.com Password: robertsmithpass123 Name on card: Robert Smith Card Number: 5513 3110 0011 1005 CVV/CVC: 777 Expiry Date: 06 / 31
Kroki	1. Przejdź na stronę główną https://automationexercise.com/ 2. Z sekcji produktów „Features Items” wybierz trzeci produkt (Sleeveless Dress) oraz wciśnij przycisk „Add to cart” 3. W wyświetlonym komunikacie naciśnij podświetlony załącznik „View Cart” 4. Kliknij przycisk „Proceed To Checkout” 5. Naciśnij przycisk „Place Order”. 6. Wprowadź dane karty płatniczej z danych wejściowych.

	7. Naciśnij przycisk „Pay and Confirm Order”
Oczekiwany rezultat	Przed przejściem procesu zamówienia użytkownikowi powinien zostać wyświetlony komunikat „Your order has been placed succesfully!” Oraz użytkownik zostaje przekierowany do podsumowania zamówienia, gdzie zostaje wyświetlona wiadomość „Order placed!” „Congratulations! Your order has been confirmed!”

Identyfikator	TC-CHECKOUT02 – Checkout zamówionych produktów jako niezalogowany użytkownik
Priorytet	Wysoki
Opis	Testowanie flow aplikacji poprzez sprawdzenie czy użytkownik może zamówić dodane do koszyka produkty.
Dane wejściowe	Niezalogowany użytkownik
Kroki	8. Przejdź na stronę główną https://automationexercise.com/ 9. Z sekcji produktów „Features Items” wybierz trzeci produkt (Sleeveless Dress) oraz wciśnij przycisk „Add to cart” 10. W wyświetlonym komunikacie naciśnij podświetlony załącznik „View Cart” 11. Kliknij przycisk „Proceed To Checkout”
Oczekiwany rezultat	Użytkownikowi zostaje wyświetlony komunikat, aby zalogować lub zarejestrować się funkcją „Register / Login”.

Identyfikator	TC-CHECKOUT03 – Checkout zamówionych produktów jako nowo zarejestrowany użytkownik
Priorytet	Najwyższy
Opis	Testowanie flow aplikacji poprzez walidację poprawnych danych podczas wymaganej rejestracji przed checkoutem.
Dane wejściowe	Dane wygenerowane przez bibliotekę Faker do zarejestrowania użytkownika.
Kroki	1. Przejdź na stronę główną https://automationexercise.com/ 2. Z sekcji produktów „Features Items” wybierz trzeci produkt (Sleeveless Dress) oraz wciśnij przycisk „Add to cart” 3. W wyświetlonym komunikacie naciśnij podświetlony załącznik „View Cart” 4. Kliknij przycisk „Proceed To Checkout” 5. W wyświetlonym komunikacie naciśnij podświetlony link „Register / Login”. 6. Wprowadź name oraz e-mail z danych testowych w pole rejestracji. 7. Zarejestruj nowego użytkownika. 8. Wciśnij przycisk „Continue” po poprawnej rejestracji. 9. Z górnego paska wybierz „Cart”
Oczekiwany rezultat	W koszyku nowo zarejestrowanego użytkownika znajduje się wcześniej dodany produkt, użytkownik może kontynuować zamówienie, a w rubryce adresu wyświetla się poprawnie wprowadzony adres z procesu rejestracji.

6. Framework i automatyzacja przypadków testowych

- **Przygotowanie struktury do testów**

W wcześniejszej części dokumentacji opisałem proces utworzenia i podstawowego skonfigurowania projektu przed testowaniem, zostały utworzone również adekwatne przypadki testowe sprawdzające główne najważniejsze moduły aplikacji.

Następnym krokiem było kompleksowe utworzenie struktury projektu z wykorzystaniem wzorca POM.

Page Object Model – Jest to wzorec projektowy wykorzystywany przy tworzeniu automatyzacji testów, jego zadaniem jest oddzielenie logiki biznesowej oraz elementów interfejsu użytkownika od implementacji samych testów.

```
//== Locators ==//
//Card details
private final Locator nameOnCard; 2 usages
private final Locator cardNumber; 2 usages
private final Locator CVCNumber; 2 usages
private final Locator expirationRateMonth; 2 usages
private final Locator expirationRateYear; 2 usages

private final Locator confirmOrderButton; 2 usages

public PaymentPage(Page page) { 1 usage  ⚡ Bartłomiej Wilk
    this.page = page;

    this.nameOnCard = page.locator(selector: "[data-qa='name-on-card']");
    this.cardNumber = page.locator(selector: "[data-qa='card-number']");
    this.CVCNumber = page.locator(selector: "[data-qa='cvc']");
    this.expirationRateMonth = page.locator(selector: "[data-qa='expiry-month']");
    this.expirationRateYear = page.locator(selector: "[data-qa='expiry-year']");

    this.confirmOrderButton = page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Pay and Confirm Order"));
}

@Step("Enter payment details (Card)") 1 usage  ⚡ Bartłomiej Wilk
public void enterCardDetails(String name, String number, String cvc, String expirationMonth, String expirationYear){
    nameOnCard.fill(name);
    cardNumber.fill(number);
    CVCNumber.fill(cvc);
    expirationRateMonth.fill(expirationMonth);
    expirationRateYear.fill(expirationYear);
}
```

Przedstawiony kod to gotowa struktura odpowiadająca stronie płatności w naszym podmiocie testów. Typowa budowa plików POM to inicjalizacja elementów UI, czyli w Playwright nazywane **Locatorami** i **Selectorami**, następnie konstruktor klasy tworzy nam przypisanie obiektu strony, zgodnie z zasadą **Single Responsibility**, czyli nasza klasa **PaymentPage** zawiera wyłącznie elementy znajdujące się na tej stronie, oraz przypisuje zadeklarowane Locatory do elementów interfejsu użytkownika.

- **Tworzenie klas testowych**

Aby odwzorować przypadki testowe w formie testów, utworzyliśmy klasy testowe dla modułów objętych zakresem testów. Dla przedstawienia procesu tworzenia klas testowych jako przykład przedstawię **RegisterTest** czyli klasę w której będą znajdować się przypadki testowe dla modułu rejestracji.

```
@ExtendWith(TestResultWatcher.class)
public class RegisterTest extends BaseTest {
```

W tym kawałku kodu możemy zobaczyć inicjację klasy testowej, następnym krokiem było dziedziczenie wcześniej utworzonej klasy `BaseTest`, w celu inicjacji ważnych zmiennych wykorzystywanych podczas testów oraz metod tworzących punkt startowy naszych testów.

Ważnym elementem jest także zapis przed deklaracją, jest to adnotacja mówiąca, że chcemy również rozszerzyć naszą klasę o klasę **TestResultWatcher**, o której dokładnie dowiemy się w dalszej części dokumentacji, ale w wielkim skrócie, jest to klasa, która monitoruje testy oraz zwraca wynik (jak pass/fail/skipped).

- **Tworzenie przypadków testowych**

Następnie z wykorzystaniem zaimplementowanych w POM metod, przekształcamy przypadki testowe z dokumentu do ich kodowej wersji.

```
@Test
public void shouldRegisterNewUserSuccessfully() {

    HomePage homePage = new HomePage(page);
    LoginPage loginPage = new LoginPage(page);
    SignupPage signupPage = new SignupPage(page);
    Topbar topbar = new Topbar(page);
    Locator accountCreated = page.locator("[data-qa='account-created']");
    Faker faker = new Faker();
    PopupUtils popupUtils = new PopupUtils(page);
    String firstName = faker.name().firstName();
    String lastName = faker.name().lastName();
    String address = faker.address().fullAddress();
    String email = faker.internet().emailAddress();
    String password = faker.credentials().password();
    Gender gender = faker.options().option(Gender.class);
    int day = faker.number().numberBetween(1, 32);
    int month = faker.number().numberBetween(1, 13);
    int year = faker.number().numberBetween(1990, 2005);
    String state = faker.address().state();
    String city = faker.address().city();
    String zipcode = faker.address().zipCode();
    String phoneNumber = faker.phoneNumber().phoneNumber();

    homePage.open();
    popupUtils.acceptCookiesIfPopupIsVisible();
    topbar.clickLogin();
    loginPage.preRegisterAndClickSignupButton(firstName, email);
    signupPage.selectGender(gender);
    signupPage.enterPassword(password);
    signupPage.enterDateOfBirth(day, month, year);
    signupPage.signupForNewsletter();
    signupPage.signupForSpecialOffers();
```



```

signupPage.enterAddressInformations(firstName, lastName, address);
signupPage.selectCountry("United States");
signupPage.enterAddressDetails(state, city, zipcode);
signupPage.enterPhoneNumber(phoneNumber);
signupPage.clickCreateAccount();

assertThat(accountCreated).isVisible();
assertThat(accountCreated).containsText("Account Created!");
}

```

Nie trzeba być specjalistą, żeby zauważyć, że wstępna implementacja przypadku testowego jest dość długa, nasz test zaczyna się kilkunastoma linijkami kodu które tworzą dane, obsługują popup, w wielkim skrócie ten test powinien badać, czy użytkownik może się zarejestrować bez problemu a nie jak zainicjować i wypełnić 17 pól formularza. Był to pierwszy utworzony przypadek testowy, i pomimo, że spełniał swoje zadanie, bo przypadek testowy dokładnie przedstawiał proces rejestracji, tak zawierał kilka złych praktyk, jak np.: powtarzalność kodu w klasie przez tworzenie Page Object w każdym teście, bezpośrednie tworzenie i używanie Locatorów w teście, oraz obniżenie czytelności poprzez bardzo krokowe wykonywanie procesu rejestracji.

```

@ExtendWith(TestResultWatcher.class)  & Bartłomiej Wilk
public class RegisterTest extends BaseTest {

    @BeforeEach  & Bartłomiej Wilk
    void navigateToLoginAndRegisterPage() {
        homepage.open();
        popupUtils.acceptCookiesIfPopupIsVisible();
        topbar.clickLogin();
    }

    @DisplayName("TC-RE601 - Poprawna rejestracja użytkownika w systemie")  & Bartłomiej Wilk
    @Feature("Register")
    @Severity(SeverityLevel.CRITICAL)
    @Test
    public void shouldRegisterNewUserSuccessfully(){
        User user = UserFactory.createRandomUser();
        Locator successMessage = signupPage.getAccountCreatedText();
        loginPage.preRegisterAndClickSignupButton(user.getFirstName(), user.getEmail());
        signupPage.completeUserRegistration(user);

        assertThat(successMessage).isVisible();
        assertThat(successMessage).containsText( expected: "Account Created!");
    }
}

```

Natomiast w późniejszym etapie tworzenia testów wygląda to już dużo lepiej i czytelniej, i właśnie takie powinny być testy w playwright, powinny być zrozumiałe aby można było je czytać jak instrukcję obsługi. Powtarzalny kod został przeniesiony do metody z adnotacją `@BeforeEach` co oznacza, że wykona się przed każdym testem. Refaktoryzacja pozwoliła na zwiększenie czytelności testów, ograniczenie duplikacji kodu oraz lepsze oddzielenie logiki biznesowej od implementacji technicznej, dzięki czemu testy stały się łatwiejsze i bardziej odporne na zmiany w interfejsie użytkownika.

- **Integracja z Allure – czyli reporty z testów**

Zielony kolor przy wykonanym teście to zawsze świetny widok, ale niestety nie jest on czasem wystarczający, aby przedstawić jako „raport z testów”. Z pomocą przychodzi nam biblioteka **Allure** – czyli pomocne narzędzie, które zbiera informację z naszego IDE oraz przedstawia w przystępnej formie.

Konfiguracja Allure przebiega tak, że potrzebujemy oznaczyć odpowiednimi adnotacjami kroki wykonywane w teście oraz test odpowiednimi wartościami. Allure pozwala użyć wielu adnotacji, które zostaną odpowiednio pogrupowane, świetne rozwiązanie, jeżeli w raporcie chcemy przedstawić różne rodzaje testów oraz moduły.

```
@DisplayName("TC-SEARCH01 - Wyszukiwanie prawidłowego produktu")
@Feature("Search")
@Severity(SeverityLevel.CRITICAL)
@Test
public void shouldSuccessfullySearchForExistingProduct() {
```

```
@Step("Click \"View Product\" on product with id {productId}") 1 usage new *
public void openViewProductPage(int productId) {
    page.navigate(url: ConfigManager.BASE_URL + "product_details/" + productId);
}
```

- **Obsługa screenshotów i trace-view**

Playwright posiada w swoich zasobach kilka ciekawych narzędzi, jednym z nich jest możliwość tworzenia zrzutów ekranów oraz nagrywania każdego kroku w teście, jest to niezwykle użyteczne w procesie debugowania testów oraz sprawdzania i eliminowania słabych testów zwanych **flaky tests**. Naszym jedynym problemem było utworzenie punktu, w którym screenshot byłby wywoływany, JUnit nie posiada możliwości sprawdzenia statusu wykonanego testu, a więc zostały utworzone dwa narzędzia, **TestResultWatcher** oraz **ScreenshotExtension**.

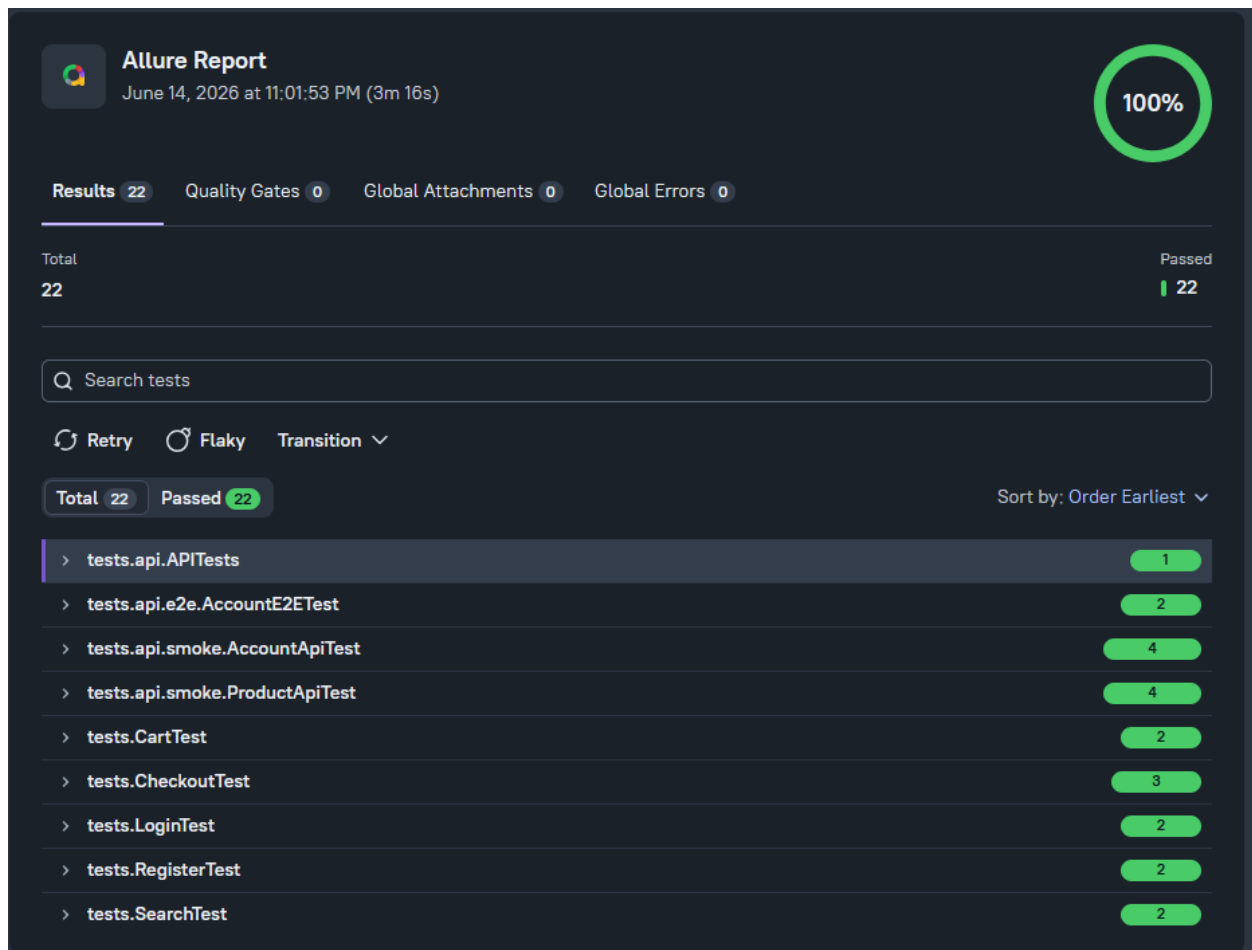
```
public class TestResultWatcher implements TestExecutionExceptionHandler
{
    @Override
    public void handleTestExecutionException(ExtensionContext context,
        Throwable throwable) throws Throwable {

        BaseTest baseTest = (BaseTest)
context.getRequiredTestInstance();

        if (baseTest.getPage() != null) {
            ScreenshotExtension.attachScreenshot(baseTest.getPage());
            ScreenshotExtension.attachCurrentUrl(baseTest.getPage());
        }

        throw throwable;
    }
}
```

Jest to klasa, która implementuje własne rozszerzenie wykorzystując interfejs JUnit 5 o nazwie **TestExecutionExceptionHandler**. Stworzona, aby obserwować wywołanie testów a następnie przechwytywać wszystkie wyjątki przez które nasze testy nie przechodzą, automatycznie tworząc zrzut ekranu oraz zapisując aktualny adres URL do raportu Allure. Korzystając z JUnit to jeden z minusów, system testów z innej biblioteki **TestNG** posiada wbudowaną funkcjonalność, który zwraca status wykonania testu, co znacząco ułatwia sprawę w którym miejscu umieścić wykonywanie zrzutu ekranu, zapisywania logów czy adresu strony.



- **Wdrażanie CI/CD do projektu**

W jakiegokolwiek formie to pojęcie występuje, prędzej czy później spotka każdego pracującego przy jakości oprogramowania, CI/CD, czyli proces ciągłej integracji i ciągłego wytwarzania, w naszym projekcie zastosowaliśmy istotny element frameworka wspierający automatyzację. Użyte narzędzie Github Actions umożliwia automatyzację procesów przy budowaniu, wdrażaniu oraz testowaniu aplikacji z poziomu repozytorium Github, eliminując konieczność manualnego uruchamiania testów po każdej zmianie w repozytorium, i usprawniając proces eliminacji błędów przy wdrażaniu zmian w naszym repozytorium.

Github Actions to idealne narzędzie, które pozwoli nam poznać proces CI/CD oraz tworzenia specyficznych pipeline'ów w naszym projekcie. Najważniejszym punktem jest utworzenie tak zwanego **Workflow**, czyli pliku z rozszerzeniem **.yaml** w którym będą znajdować się specyficzne instrukcje dla naszego repozytorium.

```
on:
  push:
    branches: [ master ]
  pull_request:
    branches: [ master ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Java
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 21

      - name: Cache Maven packages
        uses: actions/cache@v4
        with:
          path: ~/.m2
          key: ${ runner.os }-maven-${ hashFiles('**/pom.xml') }
          restore-keys: |
            ${ runner.os }-maven-

      - name: Install Playwright browsers
        run: mvn exec:java -e -Dexec.mainClass=com.microsoft.playwright

      - name: Run tests
        run: mvn clean test

      - name: Upload Allure Results
        uses: actions/upload-artifact@v4
        if: always()
```

Przygotowany pipeline uruchamiany jest automatycznie po wysłaniu zmian do głównej gałęzi repozytorium (push) oraz przy utworzeniu lub aktualizacji żądania połączenia zmian (pull request). Proces naszego pipeline składa się etapów jak:

Checkout repository – Pobieranie aktualnego kodu z naszego repozytorium.

Setup Java – Konfiguracja środowiska wykonawczego.

Cache Maven packages – Konfiguracja zapisu pobieranych zależności, aby późniejsze wywołania pipeline były szybsze.

Install Playwright browsers – Instalacja wymaganych przeglądarek oraz zależności systemowych niezbędnych do funkcjonowania Playwright.

Run Tests – Uruchomienie wszystkich testów znajdujących się w odpowiednich katalogach za pomocą polecenia **mvn clean test**.

Upload Allure Results – Archiwizacja wyników testów w postaci wygenerowanych raportów.

7. Testowanie API

W tej części zajmiemy się procesem testowania API które zostało również udostępnione w podmiocie testów. Stworzymy adekwatną strukturę do testów, przypiszemy endpointy oraz stworzymy adekwatne funkcje, które pomogą nam w tym procesie. Do testów wykorzystam udostępniony przez Playwright klient HTTP, który eliminują potrzebę importowania do projektu osobnych bibliotek jak **Retrofit** czy **RestAssured**. Do weryfikacji wyników oraz poprawności wywoływania metod użyję również narzędzia **Postman**, w którym stworzę odpowiednią kolekcję, przypiszę zmienne oraz stworzę odpowiednie metody uzupełnione o wymagane parametry.

- **Scope of testing**

Endpointy odpowiedzialne za wyszukanie, utworzenie produktów, wyszukiwanie oraz dodawanie marek.

Endpointy odpowiedzialne za utworzenie, modyfikację oraz usunięcie konta użytkownika.

- **Out of scope**

Wszelkie testy wydajnościowe oraz bezpieczeństwa, nie będziemy sprawdzać także obciążenia naszych endpointów.

- **Przypadki testowe – Smoke Testing API**

Tytuł	[TC-API01] Pobierz listę produktów
Wymagane parametry	-
Kroki do wykonania	1. Wyślij zapytanie GET na adres /api/productsList.
Oczekiwany rezultat	Status code: 200 OK Żądanie zwraca odpowiedź w postaci listy wszystkich produktów.

Tytuł	[TC-API02] Wyszukaj istniejący produkt
Wymagane parametry	Wymagane parametry: search_product
Kroki do wykonania	1. Wyślij zapytanie POST na adres /api/searchProduct z wymaganymi parametrami.
Oczekiwany rezultat	Status code: 200 OK. Żądanie zwraca odpowiedź w postaci listy z produktami wyszukiwanymi.

Tytuł	[TC-API03] Weryfikacja występowania poprawnie zarejestrowanego konta w systemie.
Wymagane parametry	Wymagane parametry: email, password
Kroki do wykonania	1. Wyślij zapytanie POST na adres /api/verifyLogin z wymaganymi parametrami.
Oczekiwany rezultat	Status code: 200 OK Żądanie zwraca wiadomość „User exists!”.

Tytuł	[TC-API04] Pobierz listę marek producentów
Wymagane parametry	-
Kroki do wykonania	1. Wyślij zapytanie GET na adres /api/brandsList.
Oczekiwany rezultat	Status code: 200 OK Żądanie zwraca odpowiedź w postaci listy wszystkich marek producentów.

Tytuł	[TC-API05] Poprawna rejestracja użytkownika w systemie
Wymagane parametry	Name, email, password, title (Mr. Ms.), birth_date, birth_month, birth_year, firstname, lastname, address1, country, zipcode, state, city, mobile_number
Kroki do wykonania	1. Wyślij zapytanie POST na endpoint /api/createAccount podając wymagane parametry.
Oczekiwany rezultat	Status code: 201 Żądanie zwraca odpowiedź z wiadomością „User created!”

- Przypadki testowe – Wyszukiwanie produktów oraz marek**

Tytuł	[TC-API06] Wyszukiwanie produktów bez podawania odpowiednich parametrów
Wymagane parametry	-
Kroki do wykonania	1. Wyślij zapytanie POST na adres /api/searchProduct
Oczekiwany rezultat	Status code: 400 Żądanie zwraca komunikat błędu „Bad request, search_product parameter is missing in POST request.

- Przypadki testowe – Autoryzacja i walidacja kont**

Tytuł	[TC-API07] Weryfikacja występowania konta w systemie bez podawania parametru
Wymagane parametry	Wymagane parametry: password
Kroki do wykonania	1. Wyślij zapytanie POST na endpoint /api/verifyLogin z wymaganym parametrem.
Oczekiwany rezultat	Status code: 400 Żądanie zwraca komunikat błędu „Bad request, email or password parameter is missing in POST request.

Tytuł	[TC-API08] Zaktualizowanie danych użytkownika bez podawania parametrów
Wymagane parametry	-
Kroki do wykonania	1. Wyślij zapytanie PUT na endpoint /api/updateAccount
Oczekiwany rezultat	Status code: 400 Żądanie zwraca komunikat błędu „Bad request, email parameter is missing in PUT request”

• Przypadki testowe – E2E

Tytuł	[TC-E2EAPI-01] Utworzenie konta użytkownika, walidacja występowania w systemie a następnie usunięcie konta
Wymagane parametry	Wymagane parametry: name, email, password, title (Mr. Ms.), birth_date, birth_month, birth_year, firstname, lastname, address1, country, zipcode, state, city, mobile_number
Kroki do wykonania	1. Wyślij zapytanie POST na endpoint /api/createAccount z wymaganymi parametrami. 2. Wyślij zapytanie POST na endpoint /api/verifyLogin z parametrami email oraz password z wymaganych parametrów. 3. Wyślij zapytanie DELETE na endpoint /api/deleteAccount z parametrami email oraz password z wymaganych parametrów.
Oczekiwany rezultat	Kroki powinny następująco zwrócić kody – 201, 200, 200 oraz użytkownik powinien otrzymać następujące wiadomości: „User created!”, „User exists!”, „Account deleted!”

Tytuł	[TC-E2EAPI-02] Utworzenie nowego konta użytkownika, zaktualizowanie istniejących danych oraz walidacja poprawności danych.
Wymagane parametry	Wymagane parametry: name, email, password, title (Mr. Ms.), birth_date, birth_month, birth_year, firstname, lastname, address1, country, zipcode, state, city, mobile_number
Kroki do wykonania	1. Wyślij zapytanie POST na endpoint /api/createAccount z wymaganymi parametrami. 2. Wyślij zapytanie PUT na endpoint /api/updateAccount z parametrami email oraz password oraz parametrem „firstname” z wartością „robUpdated” 3. Wyślij zapytanie GET na endpoint /api/getUserDetailByEmail z parametrem email.
Oczekiwany rezultat	Kroki powinny następująco zwrócić kody – 201, 200, 200 oraz użytkownik powinien otrzymać następujące wiadomości: „User created!”, „User updated!” oraz zostają wyświetlone użytkownikowi wszystkie dane wraz z zaktualizowaną wartością firstName a pozostałe wartości zostały identyczne.